

Pinocchio and Groth16 notes

arnaucube

2026

Abstract

Notes taken while studying the schemes Pinocchio ([?]) and Groth16 ([?]).

Usually while reading books and papers I take handwritten notes in a notebook, this document contains some of them re-written to *LaTeX*.

The proofs may slightly differ from the ones from the books or papers, since I try to extend them for a deeper understanding.

Contents

1	Introduction	2
2	R1CS	2
3	QAP (Quadratic Arithmetic Programs)	3

1 Introduction

2 R1CS

Rank-One Constraint System (R1CS) is the representation of the constraints of the circuit by 3 linear combinations, where there is one constraint per operation.

$$\begin{aligned}
& (a_{11}w_1 + a_{12}w_2 + \dots + a_{1n}w_n) \cdot (b_{11}w_1 + b_{12}w_2 + \dots + b_{1n}w_n) \\
& \quad - (c_{11}w_1 + c_{12}w_2 + \dots + c_{1n}w_n) = 0 \\
& (a_{21}w_1 + a_{22}w_2 + \dots + a_{2n}w_n) \cdot (b_{21}w_1 + b_{22}w_2 + \dots + b_{2n}w_n) \\
& \quad - (c_{21}w_1 + c_{22}w_2 + \dots + c_{2n}w_n) = 0 \\
& (a_{31}w_1 + a_{32}w_2 + \dots + a_{3n}w_n) \cdot (b_{31}w_1 + b_{32}w_2 + \dots + b_{3n}w_n) \\
& \quad - (c_{31}w_1 + c_{32}w_2 + \dots + c_{3n}w_n) = 0 \\
& \quad \dots \\
& (a_{m1}w_1 + a_{m2}w_2 + \dots + a_{mn}w_n) \cdot (b_{m1}w_1 + b_{m2}w_2 + \dots + b_{mn}w_n) \\
& \quad - (c_{m1}w_1 + c_{m2}w_2 + \dots + c_{mn}w_n) = 0
\end{aligned}$$

Where the R1CS linear combinations can be represented by three matrices A , B and C :

$$\begin{aligned}
A &= \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ a_{31} & a_{32} & \dots & a_{3n} \\ \dots & & & \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix} \quad B = \begin{pmatrix} b_{11} & b_{12} & \dots & b_{1n} \\ b_{21} & b_{22} & \dots & b_{2n} \\ b_{31} & b_{32} & \dots & b_{3n} \\ \dots & & & \\ b_{m1} & b_{m2} & \dots & b_{mn} \end{pmatrix} \\
C &= \begin{pmatrix} c_{11} & c_{12} & \dots & c_{1n} \\ c_{21} & c_{22} & \dots & c_{2n} \\ c_{31} & c_{32} & \dots & c_{3n} \\ \dots & & & \\ c_{m1} & c_{m2} & \dots & c_{mn} \end{pmatrix}
\end{aligned}$$

Where $(A, B, C) = Aw \circ Bw - Cw = 0$

The process that converts the *flat constraints* into *R1CS* is the following:

Lets assume that we have a constraint $5w_3 = w_1 \cdot 2w_2$ (being w_1 , w_2 and w_3 the signals 1, 2 and 3 respectively). We can represent this constraint in the matrices form by:

$$\begin{aligned}
A &= (1 \quad 0 \quad 0 \quad \dots \quad 0) \\
B &= (0 \quad 2 \quad 0 \quad \dots \quad 0) \\
C &= (0 \quad 0 \quad 5 \quad \dots \quad 0)
\end{aligned}$$

Which in the R1CS equation form would look as:

$$\begin{aligned}
& (1 \cdot w_1 + 0 \cdot w_2 + 0 \cdot w_3 + \dots + 0 \cdot w_n) \cdot (0 \cdot w_1 + 2 \cdot w_2 + 0 \cdot w_3 + \dots + 0 \cdot w_n) - \\
& (0 \cdot w_1 + 0 \cdot w_2 + 5 \cdot w_3 + \dots + 0 \cdot w_n) = 0
\end{aligned}$$

If we simplify the previous linear combination we get:

$$\begin{aligned}(1 \cdot w_1) \cdot (2 \cdot w_2) - (5 \cdot w_3) &= 0 \\ w_1 \cdot 2w_2 - 5w_3 &= 0 \\ 5w_3 &= w_1 \cdot 2w_2\end{aligned}$$

Where $5w_3 = w_1 \cdot 2w_2$ is the constraint that we wanted to represent in the R1CS format.

The nice thing from R1CS is that allows us to standarize the constraints in an equation fashion, where all the signals are involved and we 'activate' these ones that are used in each constraint. Since we have now the constraints in a polynomial format, we can do work with it in the polynomial way.

3 QAP (Quadratic Arithmetic Programs)

Once we have the constraints of a circuit represented in *R1CS*, we can do one more iteration and represent it in *Quadratic Arithmetic Programs (QAP)*. *QAP* is a representation of the XXX in a linear combination of the *R1CS*, represented by 3 polynomials $a(X)$, $b(X)$ and $c(X)$. The *QAP* consists of the 3 polynomials $a(X)$, $b(X)$ and $c(X)$, plus a divisor polynomial $h(X) \in \mathbb{F}[X]^{<d}$.

The main idea behind the *QAP* is that they 'contain' all the constraints of the circuit, meaning that these polynomials need to be able to generate all the constraints.

In order to 'collapse' the *R1CS* polynomials into the 3 polynomials of the *QAP*, we make use of *polynomial interpolation*, also named *Lagrange interpolation*. *Lagrange interpolation* allows for a group of points, to find the smallest degree and unique polynomial that goes through all those points. It can be calculated by computing:

$$L(X) = \sum_{j=0}^n y_j l_j(X)$$

where $l_j(X)$ is:

$$l_j(X) = \prod_{0 \leq m \leq k; m \neq j} \frac{X - x_m}{x_j - x_m} \dots \frac{(X - x_{j-1})(X - x_{j+1})}{(x_j - x_{j-1})(x_j - x_{j+1})} \dots \frac{X - x_k}{x_j - x_k}$$

The intuition behind the *Lagrange interpolation* is quite simple. If we have two points (on a 2 dimensional plane) we can find a line that goes through both points (this can be done with a pen and a ruler on a paper), if we have 3 points we can find a curved line that goes through all the three points, if we do this through math trying to get the lowest degree curve we obtain a 2nd degree polynomial. This can be generalized using the *Lagrange interpolation*, allowing us for a group of n points to find a $n - 1$ degree polynomial that goes through all of them.

In our case, *Lagrange interpolation* is used to go from the *R1CS* polynomials into the *QAP* polynomials, which are defined by:

$$\begin{aligned} a(X) &= (\alpha_1(X)w_1 + \alpha_2(X)w_2 + \dots + \alpha_n(X)w_n) = \sum_{i=1}^n \alpha_i(X)w_i \\ b(X) &= (\beta_1(X)w_1 + \beta_2(X)w_2 + \dots + \beta_n(X)w_n) = \sum_{i=1}^n \beta_i(X)w_i \\ c(X) &= (\gamma_1(X)w_1 + \gamma_2(X)w_2 + \dots + \gamma_n(X)w_n) = \sum_{i=1}^n \gamma_i(X)w_i \end{aligned}$$

The polynomials $a(X), b(X), c(X)$ are the QAP, such that

$$a(X) \cdot b(X) = c(X)$$

so that

$$p(X) = a(X) \cdot b(X) - c(X) = 0$$

that is,

$$\begin{aligned} p(X) &= (\alpha_1(X)w_1 + \alpha_2(X)w_2 + \dots + \alpha_n(X)w_n) \\ &\quad \cdot (\beta_1(X)w_1 + \beta_2(X)w_2 + \dots + \beta_n(X)w_n) \\ &\quad - (\gamma_1(X)w_1 + \gamma_2(X)w_2 + \dots + \gamma_n(X)w_n) \\ &= \left(\sum_{i=1}^n \alpha_i(X)w_i \right) \cdot \left(\sum_{i=1}^n \beta_i(X)w_i \right) - \left(\sum_{i=1}^n \gamma_i(X)w_i \right) \end{aligned}$$

The main idea, is that $p(X)$ defines the constraints of our circuit, so that for a valid witness evaluated at $p(X)$, it will be equal to zero.

References

- [1] Bryan Parno, Craig Gentry, Jon Howell, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. Cryptology ePrint Archive, Paper 2013/279, 2013.
- [2] Jens Groth. On the size of pairing-based non-interactive arguments. Cryptology ePrint Archive, Paper 2016/260, 2016.
- [3] <https://github.com/arnaucube/math/blob/master/weil-pairing.pdf>.